

步结束之后任何两个鬼不能占用同一个位置，也不能在一步之内交换位置。例如如图 7-17 所示的局面：一共有 4 种移动方式，如图 7-18 所示。

```
####
ab#
#c##
####
```

图 7-17 题设局面

```
####  ####  ####  ####
ab#  a b#  acb#  ab #
#c##  #c##  # ##  #c##
####  ####  ####  ####
```

图 7-18 4 种移动方式

输入保证所有空格连通，所有障碍格也连通，且任何一个 2×2 子网格中至少有一个障碍格。输出最少的步数。输入保证有解。

【分析】

以当前 3 个小写字母的位置为状态，则问题转化为图上的最短路问题。状态总数为 256^3 ，每次转移时需要 5^3 枚举每一个小写字母下一步的走法（上下左右加上“不动”）。可惜状态数已经很大了，转移代价又比较高，很容易超时，需要优化。

首先是优化转移代价。条件“任何一个 2×2 子网格中至少有一个障碍格”暗示着很多格子都是障碍，并且大部分空地都和障碍相邻，因此不是所有 4 个方向都能移动，因此可以把所有空格提出来建立一张图，而不是每次临时判断 5 种方案是否合法。加入这个优化以后 BFS 就可以通过本题的数据了，但还有改进的空间。

其次是换一个算法，例如双向广度优先搜索^①。这种算法在前面并没有介绍，但是对于“暴力搜索”这样的非常规算法来说，并不一定要严格遵守所谓的“标准方法”。例如，提到“双向广度优先算法”，可以“想当然”地设计出这样的算法：正着搜索一层，反着搜索一层，然后继续这样交替下去，直到两层中出现相同的状态，读者不妨一试。

本题非常经典，强烈推荐读者编写程序。

7.6 迭代加深搜索

迭代加深搜索是一个应用范围很广的算法，不仅可以像回溯法那样找一个解，也可以像状态空间搜索那样找一条路径。下面先举一个经典的例子。

埃及分数问题。在古埃及，人们使用单位分数的和（即 $1/a$ ， a 是自然数）表示一切有理数。例如， $2/3=1/2+1/6$ ，但不允许 $2/3=1/3+1/3$ ，因为在加数中不允许有相同的。

对于一个分数 a/b ，表示方法有很多种，其中加数少的比加数多的好，如果加数个数相同，则最小的分数越大越好。例如， $19/45=1/5+1/6+1/18$ 是最优方案。

输入整数 a, b ($0 < a < b < 500$)，试编程计算最佳表达式。

样例输入：

495 499

^① 还有一个不错的候选算法是 A*，可惜超出了本书的范围，有兴趣的读者可以自行搜索相关资料。

样例输出:

Case 1: $495/499=1/2+1/5+1/6+1/8+1/3992+1/14970$

【分析】

这道题目理论上可以用回溯法求解，但是解答树非常“恐怖”——不仅深度没有明显的上界，而且加数的选择在理论上也是无限的。换句话说，如果用宽度优先遍历，连一层都扩展不完（因为每一层都是无限大的）。

解决方案是采用迭代加深搜索（iterative deepening）：从小到大枚举深度上限 $\max d$ ，每次执行只考虑深度不超过 $\max d$ 的结点。这样，只要解的深度有限，则一定可以在有限时间内枚举到。

提示 7-17：对于可以用回溯法求解但解答树的深度没有明显上限的题目，可以考虑使用迭代加深搜索（iterative deepening）。

深度上限 $\max d$ 还可以用来“剪枝”。按照分母递增的顺序来进行扩展，如果扩展到 i 层时，前 i 个分数之和为 c/d ，而第 i 个分数为 $1/e$ ，则接下来至少还需要 $(a/b - c/d)/(1/e)$ 个分数，总和才能达到 a/b 。例如，当前搜索到 $19/45=1/5+1/100+\dots$ ，则后面的分数每个最大为 $1/101$ ，至少需要 $(19/45-1/5)/(1/101)=23$ 项总和才能达到 $19/45$ ，因此前 22 次迭代是根本不会考虑这棵子树的。这里的关键在于：可以估计至少还要多少步才能出解。

注意，这里的估计都是乐观的，因为用了“至少”这个词。说得学术一点，设深度上限为 $\max d$ ，当前结点 n 的深度为 $g(n)$ ，乐观估价函数为 $h(n)$ ，则当 $g(n)+h(n)>\max d$ 时应该剪枝。这样的算法就是 IDA*。当然，在实战中不需要严格地在代码里写出 $g(n)$ 和 $h(n)$ ，只需要像刚才那样设计出乐观估价函数，想清楚在什么情况下不可能在当前的深度限制下出解即可。

提示 7-18：如果可以设计出一个乐观估价函数，预测从当前结点至少还需要扩展几层结点才有可能得到解，则迭代加深搜索变成了 IDA* 算法。

本题的主框架就是一个简单循环：

```
int ok = 0;
for(maxd = 1; ; maxd++) {
    memset(ans, -1, sizeof(ans));
    if(dfs(0, get_first(a, b), a, b)) { ok = 1; break; }
}
```

其中 $\text{get_first}(a, b)$ 是满足 $1/c \leq a/b$ 的最小 c 。迭代加深搜索过程如下（约分的原理详见第 10 章）：

```
//如果当前解 v 比目前最优解 ans 更优，更新 ans
bool better(int d) {
    for(int i = d; i >= 0; i--) if(v[i] != ans[i]) {
        return ans[i] == -1 || v[i] < ans[i];
    }
}
```

```

    }
    return false;
}

//当前深度为 d, 分母不能小于 from, 分数之和恰好为 aa/bb
bool dfs(int d, int from, LL aa, LL bb) {
    if(d == maxd) {
        if(bb % aa) return false; //aa/bb 必须是埃及分数
        v[d] = bb/aa;
        if(better(d)) memcpy(ans, v, sizeof(LL) * (d+1));
        return true;
    }
    bool ok = false;
    from = max(from, get_first(aa, bb)); //枚举的起点
    for(int i = from; ; i++) {
        //剪枝: 如果剩下的 maxd+1-d 个分数全部都是 1/i, 加起来仍然不超过 aa/bb, 则无解
        if(bb * (maxd+1-d) <= i * aa) break;
        v[d] = i;
        //计算 aa/bb - 1/i, 设结果为 a2/b2
        LL b2 = bb*i;
        LL a2 = aa*i - bb;
        LL g = gcd(a2, b2); //以便约分
        if(dfs(d+1, i+1, a2/g, b2/g)) ok = true;
    }
    return ok;
}

```

例题 7-10 编辑书稿 (Editing a Book, UVa 11212)

你有一篇由 n ($2 \leq n \leq 9$) 个自然段组成的文章, 希望将它们排列成 $1, 2, \dots, n$ 。可以用 Ctrl+X (剪切) 和 Ctrl+V (粘贴) 快捷键来完成任务。每次可以剪切一段连续的自然段, 粘贴时按照顺序粘贴。注意, 剪贴板只有一个, 所以不能连续剪切两次, 只能剪切和粘贴交替。

例如, 为了将 {2,4,1,5,3,6} 变为升序, 可以剪切 1 将其放到 2 前, 然后剪切 3 将其放到 4 前。再如, 对于排列 {3,4,5,1,2}, 只需一次剪切和一次粘贴即可——将 {3,4,5} 放在 {1,2} 后, 或者将 {1,2} 放在 {3,4,5} 前。

【分析】

本题是典型的状态空间搜索问题, “状态”就是 $1 \sim n$ 的排列, 初始状态是输入, 终止状态是 $1, 2, 3, \dots, n$ 。因为 $n \leq 9$, 排列最多有 $9! = 362880$ 个。虽然这个数字不算大, 但是每个状态的后继状态也比较多 (有很多剪切和粘贴的方式), 所以仍有超时的危险。比赛时很多选手使用了一些“加速策略”。

策略 1: 每次只剪切一段连续的数字。例如, 不要剪切 2 4 这样数字不连续的片段。